

Introducción a Hibernate & JPA

Hibernate es un *Object-Relational Mapper* (ORM). Como su nombre indica, es quien mapea (establece una función biyectiva) entre el mundo de objetos y el de la base de datos relacional. Estos mundos, como se vió en clase, tienen profundas diferencias, y como consecuencia, hay cosas que no pueden ser mapeadas en forma completa o directa, debiendo hacerse trade-offs y seguirse convenciones. Este choque de concepto entre ambos mundos, es denominado *Impedancia Objeto-Relacional*.

Hibernate es el ORM más popular a la fecha, pero no el único. Es prácticamente quien ha definido el concepto de ORM moderno, y no por nada, el standard de Java para persistencia (JPA) está fuertemente basado en Hibernate.

Nosotros, vamos a usar JPA, con Hibernate como implementación concreta del tandard (así como usabamos el standard de validación de beans con la implementación concreta de Hibernate, ver clase de *Web Forms*).

Comenzamos a configurar Hibrenate en reemplazo de jdbc en nuestro proyecto.

Nuevas depenendecias en el pom padre:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-orm</artifactId>
  <version>${org.springframework.version}</version>
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>${org.hibernate.version}</version>
</dependency>
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-entitymanager</artifactId>
  <version>${org.hibernate.version}</version>
</dependency>
<dependency>
  <groupId>org.hibernate.javax.persistence</groupId>
  <artifactId>hibernate-jpa-2.1-api</artifactId>
  <version>${org.hibernate.jpa.version}</version>
</dependency>
```

donde:

```
<org.hibernate.version>5.1.0.Final</org.hibernate.version>
<org.hibernate.jpa.version>1.0.0.Final</org.hibernate.jpa.version>
```

en persistence:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-orm</artifactId>
</dependency>
<dependency>
```

```

    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
</dependency>
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-entitymanager</artifactId>
</dependency>

```

en model:

```

<dependency>
    <groupId>org.hibernate.javax.persistence</groupId>
    <artifactId>hibernate-jpa-2.1-api</artifactId>
</dependency>

```

en webapp:

```

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-orm</artifactId>
</dependency>

```

Ejecutamos `mvn eclipse:eclipse` y refrescamos el proyecto en Eclipse.

Con las nuevas dependencias, procedemos a actualizar la configuración de la aplicación. Editamos así `WebConfig` para definir nuestro entity manager usando JPA, y Hibernate como implementación concreta.

```

@Bean
public LocalContainerEntityManagerFactoryBean entityManagerFactory() {
    final LocalContainerEntityManagerFactoryBean factoryBean = new
LocalContainerEntityManagerFactoryBean();
    factoryBean.setPackagesToScan("ar.edu.itba.model");
    factoryBean.setDataSource(dataSource());

    final JpaVendorAdapter vendorAdapter = new HibernateJpaVendorAdapter();
    factoryBean.setJpaVendorAdapter(vendorAdapter);

    final Properties properties = new Properties();
    properties.setProperty("hibernate.hbm2ddl.auto", "update");
    properties.setProperty("hibernate.dialect",
"org.hibernate.dialect.PostgreSQL92Dialect");

    // Si ponen esto en prod, hay tabla!!!
    properties.setProperty("hibernate.show_sql", "true");
    properties.setProperty("format_sql", "true");

    factoryBean.setJpaProperties(properties);

    return factoryBean;
}

```

Cambiamos acorde el transaction manager por uno que entienda de JPA:

```

@Bean
public PlatformTransactionManager transactionManager(final EntityManagerFactory emf) {
    return new JpaTransactionManager(emf);
}

```

databasePopulator y databaseInitializer pueden eliminarse de momento, mientras dejamos que hibernate administre la base de datos. Si estuviésemos usandolos para otra cosa además del setup de la base de datos, podríamos mantenerlos.

Procedemos a armar nuestra implementación de UserDao con JPA.

```

@Repository
public class UserHibernateDao implements UserDao {

    @PersistenceContext
    private EntityManager em;

    @Override
    public User create(final String username, final String password) {
        final User user = new User(username, password);
        em.persist(user);
        return user;
    }

    @Override
    public User getByUsername(final String username) {
        final TypedQuery<User> query = em.createQuery("from User as u where u.username = :username", User.class);
        query.setParameter("username", username);
        final List<User> list = query.getResultList();
        return list.isEmpty() ? null : list.get(0);
    }

    @Override
    public User findById(long id) {
        return em.find(User.class, id);
    }
}

```

Notese que la query no está escrita en SQL, sino en JQL (JPA Query Language, a su vez basado en HQL, Hibernate Query Language). En vez de nombres de tabla usamos nombres de entidades, y en lugar de columnas usamos nombres de propiedades.

Quitamos el @Repository de UserJdbcDao para evitar encuentre dos candidatos para el mismo bean.

Y finalmente agregamos los annotations al modelo para mapearlo con JPA.

```

@Entity
@Table(name = "users")
public class User {

```

```

        @Id
        @GeneratedValue(strategy = GenerationType.SEQUENCE, generator =
"users_userid_seq")
        @SequenceGenerator(sequenceName = "users_userid_seq", name =
"users_userid_seq", allocationSize = 1)
        @Column(name = "userid")
        private Long id;

        @Column(length = 100, nullable = false, unique = true)
        private String username;

        @Column(length = 100, nullable = false)
        private String password;

        /* package */ User() {
            // Just for Hibernate, we love you!
        }

        ...

```

Hibernate **requiere** un constructor default para poder crear instancias de nuestros modelos y luego por reflection popular las properties.

El uso de `@Table` no es obligatorio, salvo que querramos evitar algún default. En este caso, queremos que se siga usando la tabla `users` en lugar de la que generaría por default `User` (notese el casing).

Análogamente, el `@Column` en la columna `id` responde a que no queremos usar el nombre default, sino que tenga el `@Id` alcanzaría.

Por defecto, Hibernate en PostgreSQL crea una única secuencia, llamada `hibernate_sequence` y la utiliza para todas las entidades. Nosotros ya teníamos una secuencia `users_userid_seq` para los ids de esta tabla. Deseamos seguir usandola, por tanto debemos agregar el atributo `generator` a `@GeneratedValue` y agregar la annotation `@SequenceGenerator` indicándole cuál es el generator que queremos usar.

Al correr la aplicación, vemos que tenemos un error, producto de que Hibernate está trayendo una dependencia transitiva que entra en conflicto con otra preexistente. Tras un breve análisis del problema concluimos que debemos modificar el pom padre para excluir la dependencia conflictiva.